



UNIVERSITÀ DI PISA

MASTER OF SCIENCE IN COMPUTER ENGINEERING

ELECTRONICS AND COMMUNICATIONS SYSTEMS

Perceptron

Project members:

Dario BANDECCHI

Contents

1	Introduction	1
1.1	Algorithm description	1
1.2	Possible implementations	3
2	Architecture description	4
2.1	Parallel Multiplier	5
2.1.1	Parallel Unsigned Multiplier	6
2.2	Adder Tree	7
2.3	Activation Function	8
3	VHDL code	9
3.1	Products.vhd	9
3.2	AdderTree.vhd	10
3.3	ActivationFunction.vhd	11
3.4	Perceptron.vhd	12
4	Test-plan	14
4.1	Ripple Carry Adder Testbench	14
4.2	Parallel Multiplier Testbench	14
4.3	Activation Function Testbench	14
4.4	Perceptron Wrapper Testbench	15
5	Vivado report	16
5.1	Synthesis and Implementation warnings	16
5.1.1	Synthesis warning explanation	16
5.1.2	Implementation warnings explanation	16
5.2	Timing constraint and Critical Path	17
5.3	Power Consumption	18
5.4	Resource Utilization	19
6	Conclusion	20

1 Introduction

1.1 Algorithm description

The perceptron is one of the first and elementary models of artificial neural networks. It is extremely simple compared to the modern standard deep learning models; however, the main concepts used in its design are also applied nowadays in the more sophisticated deep learning architectures.

In particular, the perceptron is a binary classification algorithm for supervised learning, meaning it classifies the input data according to a training procedure performed on previous inputs. A binary classifier is a function which can decide whether or not an input, represented by a vector of numbers, belongs to some specific class. It is a type of linear classifier, i.e., a classification algorithm that makes its predictions based on a linear predictor function that combines a set of weights with the feature vector.

Therefore, the perceptron takes a series of scalar features in input, along with a constant "bias", and assigns weights to these inputs. Then, it computes a linear combination of the weights and the inputs, that will be processed by an activation function, which determines the class to which the set of inputs belongs to.

There are many possible alternatives for the activation function, but the most common ones are:

- the **Heaviside function**:

$$H(x) = \begin{cases} 0 & \text{if } x < 0 \\ 1 & \text{if } x \geq 0 \end{cases} \quad (1)$$

- the **sign function**:

$$\text{sgn}(x) = \begin{cases} -1 & \text{if } x < 0 \\ 0 & \text{if } x = 0 \\ 1 & \text{if } x > 0 \end{cases} \quad (2)$$

- a **Sigmoid function**, such as $\sigma(x) = \frac{1}{1+e^{-x}}$, which has the following shape:

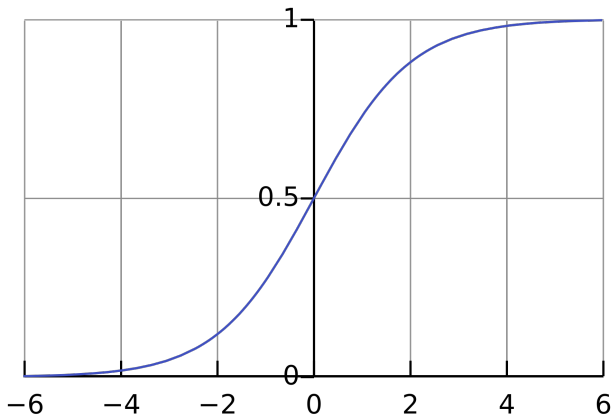


Figure 1: Sigmoid function

In our case the perceptron takes $N_{IN} = 10$ inputs x_n represented with $b_x = 8$ bits, N_{IN} weights w_n represented with $b_w = 9$ bits, and the bias b still represented with $b_w = 9$ bits. All of them shall be considered in the range $[-1,1)$ using fixed-point arithmetic.

The network carries out the product between x_n and w_n and adds the result with the bias.

This product goes in input to the activation function and the output of the system must be represented with $b_{out} = 16$ bits, truncating the least significant bits.

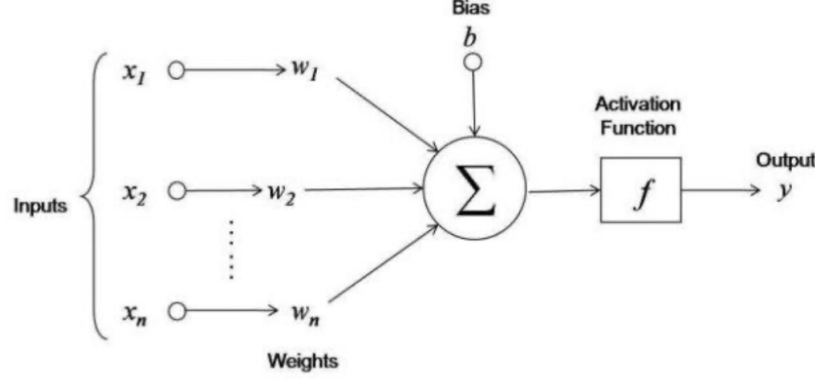


Figure 2: Perceptron block diagram

Figure 2 shows the block diagram of the perceptron algorithm that must be implemented, and its function can be summarized by the following formula:

$$y = f\left(\sum_{i=1}^{N_{IN}} x_i \cdot w_i + b\right)$$

Regarding our implementation, the activation function $f(x)$ is the following:

$$f(x) = \begin{cases} 0 & \text{if } x < -2 \\ 1/4 \cdot x + 1/2 & \text{if } -2 \leq x \leq 2 \\ 1 & \text{if } x > 2 \end{cases} \quad (3)$$

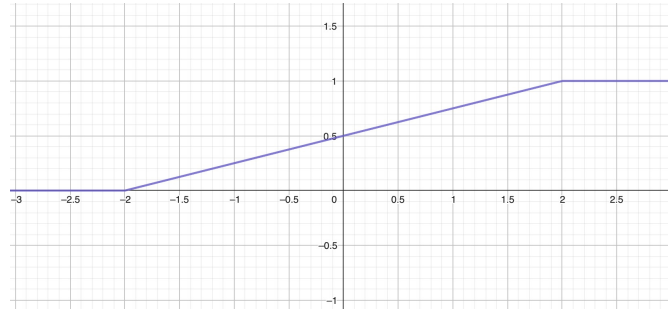


Figure 3: Activation Function

1.2 Possible implementations

To design the circuit described above, it is necessary to implement the basic blocks for the operations required, such as the products, the sums and the activation function. There are several ways to implement these blocks, notably:

- To perform the products we need a **Multiplier**, and I decided to exploit the **Parallel Multiplier** solution. Otherwise, another option could have been to use a **LUT**, but it would have been too complex and slow because of its large size (2^{17} words of 17 bits). The latter would have been suitable if the operands had fewer bits.
- As regards the sums, we need an **Adder**, and also in this case there are more than one potential solutions. The one I preferred to adopt is the **Ripple Carry Adder** implementation, because of its simplicity and efficiency at the same time. Other solutions could be the **Serial Adder**, but it is pretty slow since it requires N clock cycles to perform a sum between two N-bit words, or the **Carry Look Ahead Adder**, which is faster than the others but it requires more additional complexity in the architecture.
- To implement the **Activation function** one could have chosen to exploit a **LUT** even in this case, however, as the specifications suggest, I opted for another solution which tests the amplitude of the input and assigns the appropriate value to the output, according to the operating region in which it falls.

2 Architecture description

The overall architecture of the perceptron implementation is reported in the following block diagram:

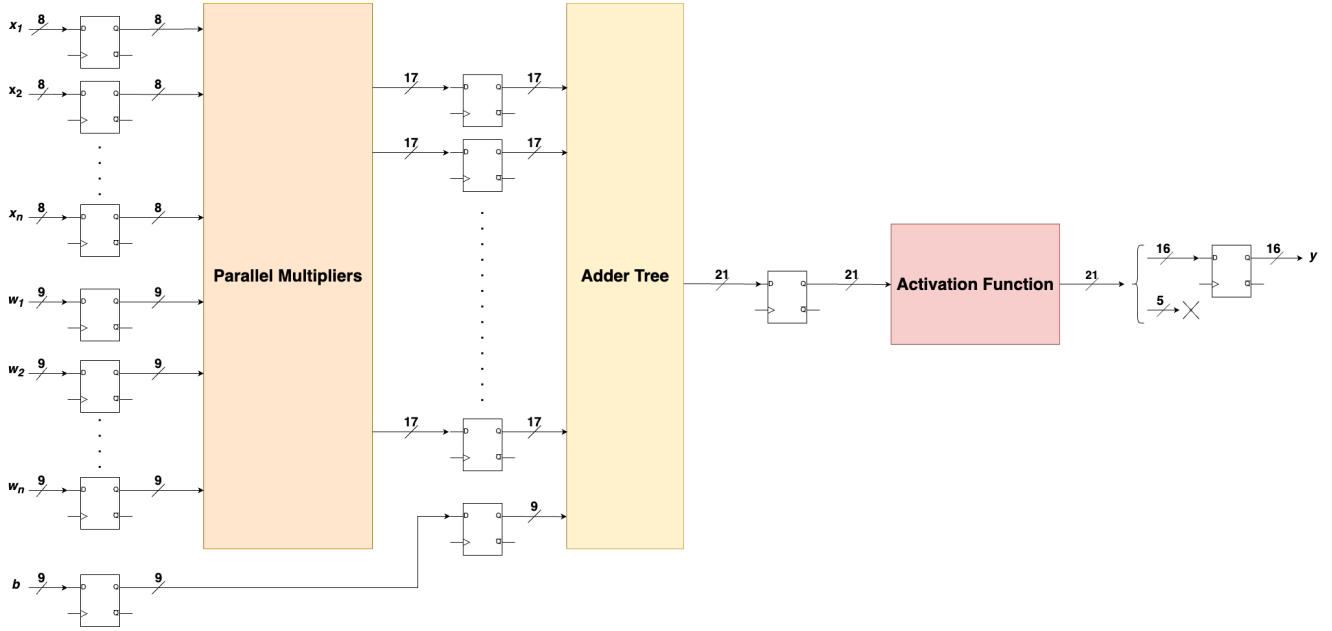


Figure 4: Perceptron overall architecture

In particular, we can notice that the main three blocks are the **Parallel Multipliers**, the **Adder Tree** and the **Activation Function**, which are described in detail in the next paragraphs.

The output of the activation function is truncated to 16 bits (by deleting its least significant bits), as required by the specifications.

Furthermore, there are some registers: the barrier ones that allow Vivado to evaluate properly all the **R-L-R paths** within the circuit, and the pipeline registers between the blocks with the aim of improving the maximum achievable clock frequency.

To avoid cluttering the figure, it was decided not to include the clock and reset signals. Clearly, these signals are fed as input to each register.

2.1 Parallel Multiplier

The Parallel Multipliers block performs the products between all the input words x_i and their respective weights w_i . Specifically, each product is computed by a dedicated **Parallel Multiplier**, whose architecture is reported in the figure below:

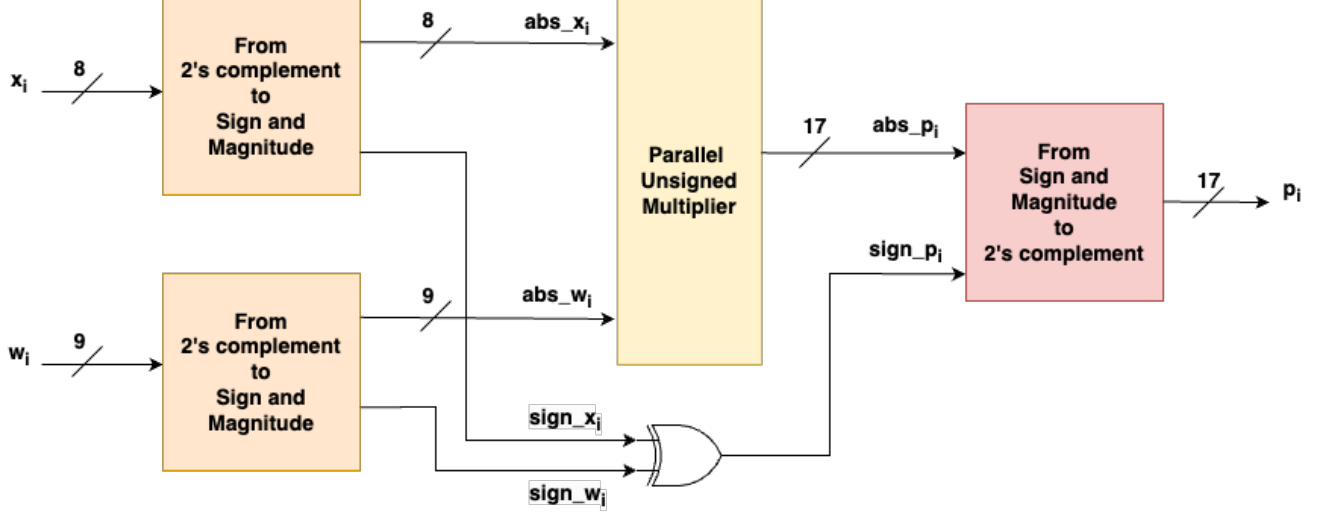


Figure 5: Parallel Multiplier architecture

As shown in Figure 5, the first step is converting the inputs from the 2's complement representation to the Sign and Magnitude one. This conversion allows us to extract both their absolute values and their signs. The former are used to compute the product's magnitude through the **Parallel Unsigned Multiplier** block (described in detail hereafter). Instead, the signs are used to compute the product's sign through the **XOR** gate.

Finally, the last block takes the product's absolute value and sign as inputs and converts it back from Sign and Magnitude to 2's complement representation.

2.1.1 Parallel Unsigned Multiplier

The Parallel Unsigned Multiplier block performs the multiplication between unsigned numbers in a parallel way, as the name suggests. It achieves this by simply exploiting few simple components, such as the **Half Adder** and **Full Adder**, in addition to some **AND** gates to compute all the partial products between the bits of the input words.

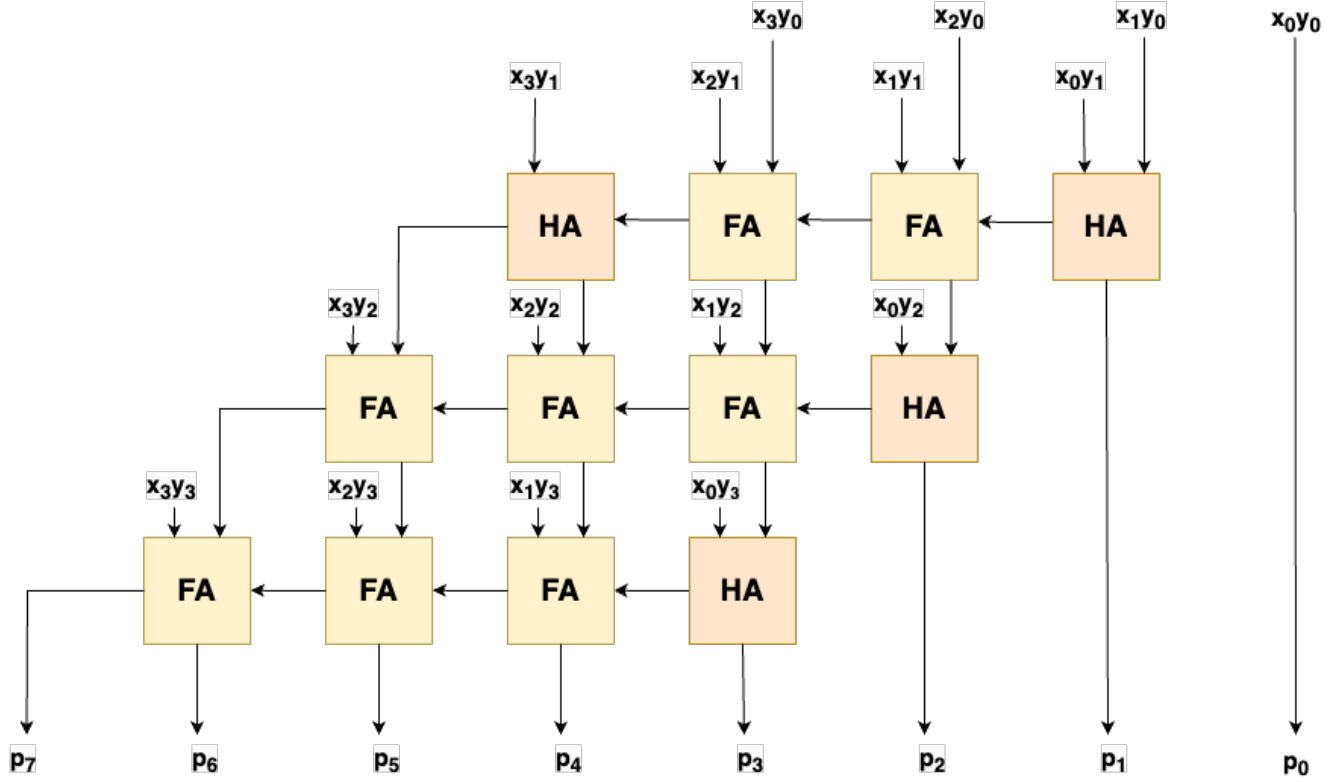


Figure 6: Parallel Unsigned Multiplier architecture

Figure 6 shows an example implementation of a 4-bit parallel unsigned multiplier for simplicity (where both input words are represented on 4 bits).

However, for our purposes, it has been used the version with, respectively, 8-bit and 9-bit input words.

2.2 Adder Tree

This block takes the outputs of the Parallel Multipliers block as inputs and performs their sum also with the bias, as shown in Figure 7:

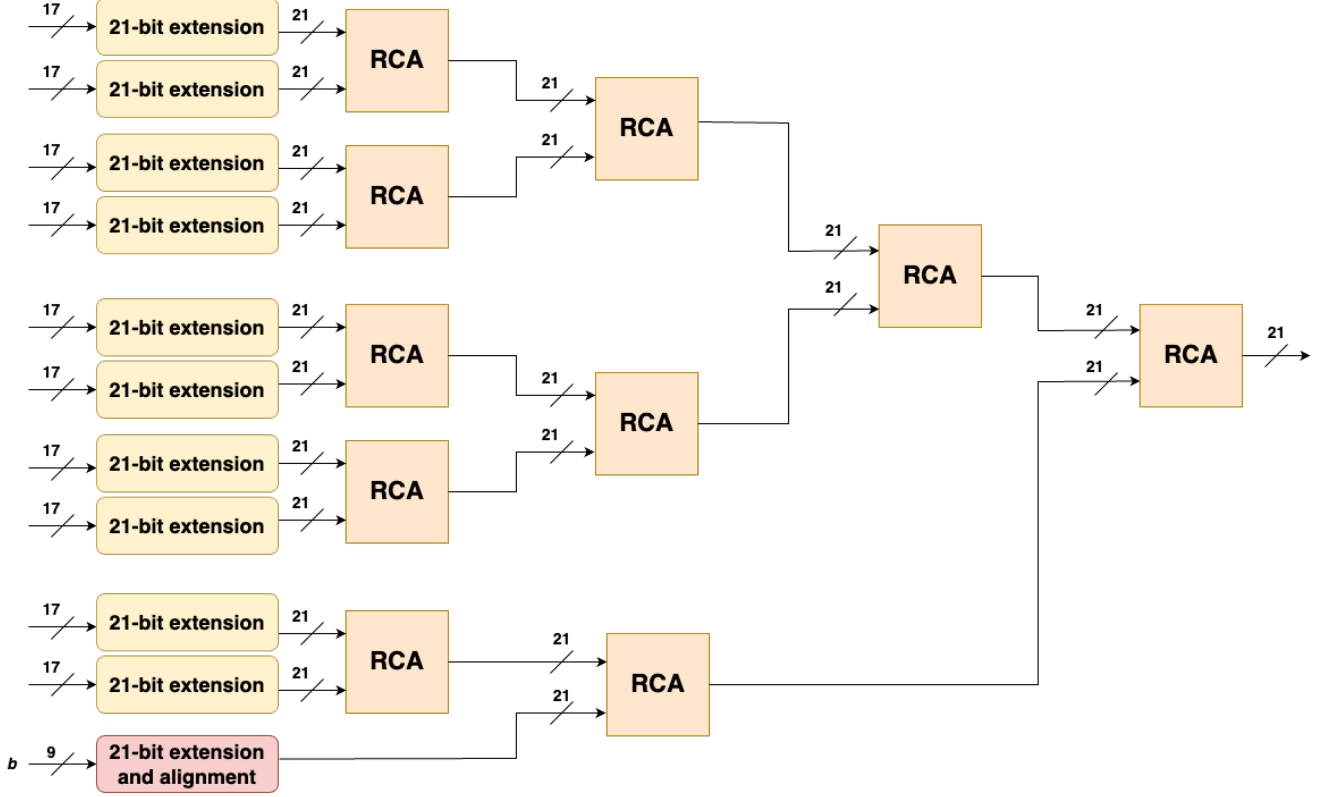


Figure 7: Adder Tree architecture

First, the inputs are extended to 21 bits to avoid sum overflows. As regards the bias, it also needs to be aligned, since it has a different **resolution** w.r.t. the other inputs. The latter operation is necessary to perform the sums properly, since all the operands must have the same *LSB*.

In particular, the other inputs have a $LSB_i = LSB_x \cdot LSB_w = 2^{-7} \cdot 2^{-8} = 2^{-15}$, meanwhile the bias has a $LSB_b = 2^{-8}$. Therefore, we have to multiply the bias by 2^7 in order to align it.

Once all the inputs are extended and aligned properly, they are ready to be processed by the RCA tree, which produces the overall sum on 21 bits.

2.3 Activation Function

The last block of the perceptron's architecture is the Activation Function, which implements the function described in the equation 3. Down below its implementation is reported:

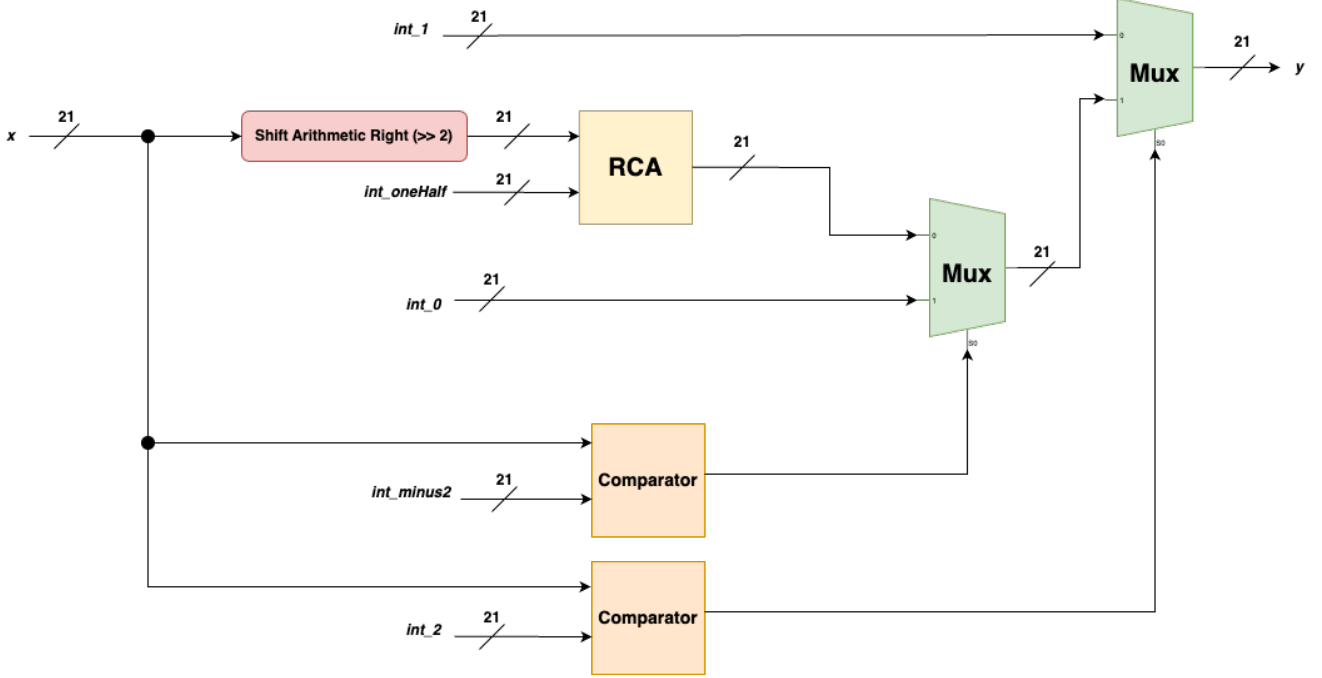


Figure 8: Activation Function architecture

The function is simply realized by just comparing the input with the integer representations of 2 and -2, in order to state which is the operating region of the Activation Function, and the output will be set accordingly (with the help of two multiplexers).

In the case of linear region, the output is computed by only shifting the input twice to the right (which corresponds to divide it by 4 efficiently) and summing it with the integer representation of $1/2$.

In the other two operating regions, the output is simply assigned to the integer representation of 0 in one case, or the integer representation of 1 in the other.

All the integer representations mentioned above can be easily computed by divided them by 2^{-15} , which is the *LSB* of x .

3 VHDL code

The circuit illustrated in Figure 4 has been implemented using VHDL code following a modular approach. Specifically, the code is organized in multiple files, each containing the implementation of a single functional block, such as the **Parallel Multiplier**, the **Adder Tree** and the **Activation Function**.

Finally, all these components are integrated into the top-module file, which implements the overall architecture.

Moreover, an additional VHDL file has been created (**myPackage.vhd**), which contains all the constants and the user-defined type declarations, and it is included in all the other files that make use of them.

The codes for the main components are described in detail below:

3.1 Products.vhd

This file implements all the products between the inputs x_i and the weights w_i , by instantiating N_{IN} Parallel Multipliers, which perform the single multiplications.

```
1 library IEEE;
2 use IEEE.std_logic_1164.all;
3 use work.myPackage.all;
4
5 entity Products is
6     port (
7         inputs    : in  inputs;
8         weights    : in  weights;
9         outputs    : out inputsAdder
10    );
11 end entity;
12
13 ...
14
15 begin
16
17     -- generate statement for all the parallel multipliers
18     g_M: for i in 0 to Nin - 1 generate
19
20         PM_i: ParallelMultiplier
21         generic map (
22             Nbit => bx,
23             Mbit => bw
24         )
25         port map (
26             a => inputs(i),
27             b => weights(i),
28             c => outputs(i)
29         );
30
31     end generate;
32
33 end architecture;
```

Listing 1: Products.vhd file

3.2 AdderTree.vhd

This file implements the **Adder Tree** architecture, as shown in Figure 7. In particular, first, all the inputs are extended to 21 bits. Then the bias is extended and aligned to fit the required accuracy. Finally, the RCAs are instantiated to perform the overall sum within the Adder Tree structure.

```
1 use IEEE.std_logic_1164.all;
2 use ieee.numeric_std.all;
3 use work.myPackage.all;
4
5 entity AdderTree is
6     port (
7         inputs    : in  inputsAdder;
8         b          : in  std_logic_vector(bw - 1 downto 0);
9         output     : out std_logic_vector(bsum - 1 downto 0)
10    );
11 end entity;
12 ...
13
14 begin
15
16     -- extension of the inputs to bsum bits
17     extendedInputs(0) <= std_logic_vector(resize(signed(inputs(0)), bsum));
18     ...
19     extendedInputs(9) <= std_logic_vector(resize(signed(inputs(9)), bsum));
20
21     -- extension and alignment of the bias
22     -- by shifting it of 7 positions to the left
23     -- and replicating 5 times the MSB
24     extendedBias <= b(b'left) & b(b'left) & b(b'left) & b(b'left) & b(b'left)
25     & b & (6 downto 0 => '0');
26
27     -- first level of the adder tree
28     RCA_1_1: RippleCarryAdder
29     generic map(
30         Nbit => bsum
31     )
32     port map(
33         a => extendedInputs(0),
34         b => extendedInputs(1),
35         cin => '0',
36         s => outputFirstLevel1,
37         cout => open
38     );
39     ...
40
41     -- fourth (last) level
42     RCA_4_1: RippleCarryAdder
43     generic map(
44         Nbit => bsum
45     )
46     port map(
47         a => outputThirdLevel,
48         b => outputSecondLevel3,
49         cin => '0',
50         s => output,
```

```

186         cout => open
187     );
188
189 end architecture;

```

Listing 2: AdderTree.vhd file

3.3 ActivationFunction.vhd

This file implements the activation function, according to the architecture explained earlier in Figure 8.

Specifically, the comparisons are carried out through the standard operators provided by the **IEEE.numeric.std** library, while the other operations have been implemented manually.

```

1 library IEEE;
2 use IEEE.std_logic_1164.all;
3 use IEEE.numeric_std.all;
4 use work.myPackage.all;
5
6 entity ActivationFunction is
7     port (
8         x : in  std_logic_vector(bsum - 1 downto 0);
9         y : out std_logic_vector(bsum - 1 downto 0)
10    );
11 end entity;
12
13 ...
14
38 begin
39
40     x_int <= to_integer(signed(x));
41
42     -- multiplexer
43     y <= std_logic_vector(to_signed(int_1, bsum)) when x_int > int_2 else
44         std_logic_vector(to_signed(0, bsum)) when x_int < int_minus2 else
45         sum_signal;
46
47     -- shifting x (with sign) of 2 positions to the right
48     shifted_signal <= x(x'left) & x(x'left) & x(bsum - 1 downto 2);
49
50     -- storing the value of the second addendum (1/2)
51     b_signal <= std_logic_vector(to_signed(int_oneHalf, bsum));
52
53     -- rippleCarryAdder to perform the sum
54     -- between the shifted_signal and 1/2
55     RCA: RippleCarryAdder
56     generic map (
57         Nbit => bsum
58     )
59     port map (
60         a => shifted_signal,
61         b => b_signal,
62         cin => '0',
63         s => sum_signal,
64         cout => open
65     );

```

```

66
67 end architecture;

```

Listing 3: ActivationFunction.vhd file

3.4 Perceptron.vhd

These main blocks are finally instantiated and connected together in the top-module file, which also includes some **pipeline registers** between them in order to achieve a higher clock frequency. The last operation is the output truncation to 16 bits, as required by the specifications.

```

1  library IEEE;
2  use IEEE.std_logic_1164.all;
3  use work.myPackage.all;
4
5  entity Perceptron is
6      port (
7          clk      : in  std_logic;
8          reset    : in  std_logic;
9          xn       : in  inputs;
10         wn       : in  weights;
11         b        : in  std_logic_vector(bw - 1 downto 0);
12         y        : out std_logic_vector(bout - 1 downto 0)
13     );
14 end entity;
15
16 ...
63 begin
64
65     Prod: Products
66     port map (
67         inputs => xn,
68         weights => wn,
69         outputs => outputsProduct_signal
70     );
71
72     -- pipeline registers
73     g_DFF: for i in 0 to Nin - 1 generate
74         PR_1: DFFN
75         generic map (
76             Nbit => bpro
77         )
78         port map (
79             clk => clk,
80             resetn => reset,
81             en => '1',
82             di => outputsProduct_signal(i),
83             do => inputsAdder_signal(i)
84         );
85     end generate;
86
87     PR_bias: DFFN
88     generic map (
89         Nbit => bw
90     )

```

```

91  port map (
92      clk => clk,
93      resetn => reset,
94      en => '1',
95      di => b,
96      do => bias_signal
97  );
98
99  AT: AdderTree
100  port map (
101      inputs => inputsAdder_signal,
102      b => bias_signal,
103      output => outputAdder_signal
104  );
105
106  -- pipeline register
107  PR_2: DFFN
108  generic map (
109      Nbit => bsum
110  )
111  port map (
112      clk => clk,
113      resetn => reset,
114      en => '1',
115      di => outputAdder_signal,
116      do => inputAF_signal
117  );
118
119  AF: ActivationFunction
120  port map (
121      x => inputAF_signal,
122      y => outputAF_signal
123  );
124
125  -- truncating the least significant bits
126  y <= outputAF_signal(bsum - 1 downto (bsum - bout));
127
128  end architecture;

```

Listing 4: Perceptron.vhd file

In the end, the Perceptron entity is encapsulated within the **PerceptronWrapper.vhd** file, which also includes the barrier registers of all the inputs and outputs.

4 Test-plan

To evaluate the behavior and effectiveness of my perceptron implementation, I developed a set of testbenches to rigorously test the key components of the architecture and simulated them using ModelSim.

In particular, first, I tested and verified the basic blocks of the circuit, such as the **Ripple Carry Adder** and the **Parallel Multiplier**, to then move on the more complex ones, like the **Activation Function**. Finally, I validated the overall architecture, namely the **Perceptron Wrapper** file.

4.1 Ripple Carry Adder Testbench

Since this module performs the sum between both signed and unsigned numbers, I decided to test it in both the scenarios.

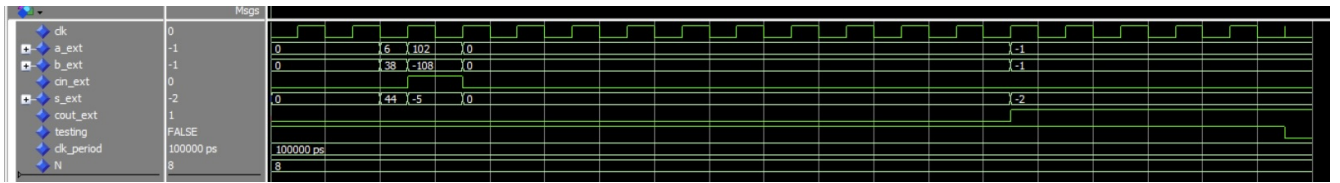


Figure 9: RCA testbench

As shown in Figure 9, this module performs properly the sums with both the representations, hence we can validate it successfully.

4.2 Parallel Multiplier Testbench

The same considerations hold also in this case, since the Parallel Multiplier must be able to carry out the products with both unsigned and signed numbers.

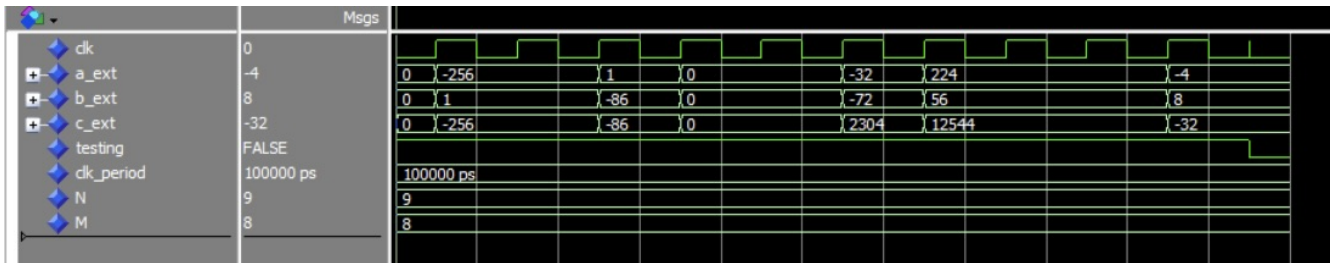


Figure 10: Parallel Multiplier testbench

As we can notice in Figure 10, we can once again successfully verify the correctness of the module.

4.3 Activation Function Testbench

As far as the validation of the activation function is concerned, I decided to assess its correctness through the verification of all the three operating regions in which it can work.

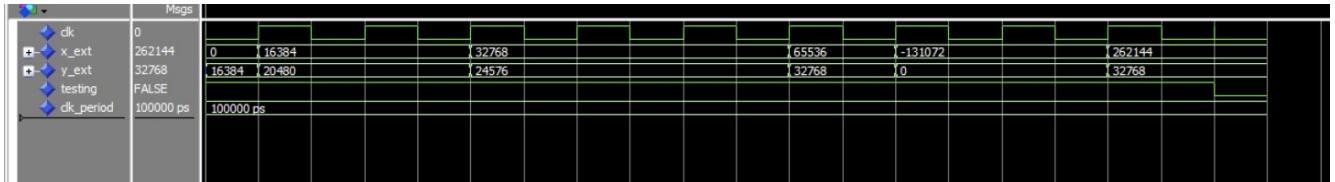


Figure 11: Activation Function testbench

Since we are working with fixed-point numbers, the above integer values can be converted to real ones by multiplying them by their $LSB = 2^{-15}$, thus one can easily confirm the correctness of the results achieved.

4.4 Perceptron Wrapper Testbench

Finally, once we have rigorously ascertained that all the basic components work properly, we can move on to verify the correctness of the overall architecture.

In particular, even in this case, I opted to test the Perceptron in all the three operating regions, by providing the appropriate inputs.

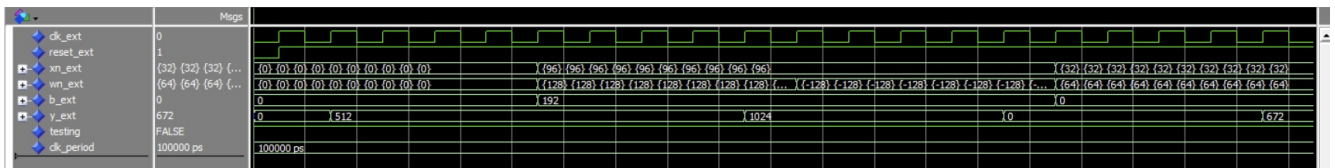


Figure 12: Perceptron Wrapper testbench

Keeping in mind once again that we are working with fixed-point representations, we can simply convert the inputs to their respective real values (by multiplying them by their *LSBs*), performing all the operations of the perceptron with the real numbers and, at the end, converting back the results in their integer representation, in order to readily see whether or not they correspond to the results provided by the testbench.

In this case, the results appear in the output only after 4 clock cycles from the moment the inputs are provided, due to the presence of 4 levels of registers.

To prove the correctness of the results, the computations for the second set of inputs are shown in detail below:

$$\begin{aligned}
y &= f \left(b \cdot LSB_b + \sum_{i=1}^{N_{IN}} (x \cdot LSB_x) \cdot (w \cdot LSB_w) \right) \\
&= f \left(192 \cdot 2^{-8} + \sum_{i=1}^{N_{IN}} (96 \cdot 2^{-7}) \cdot (128 \cdot 2^{-8}) \right) \\
&= f(0.75 + 10 \cdot (0.75 \cdot 0.5)) \\
&= f(4.5) = 1
\end{aligned} \tag{4}$$

Now, by converting y to its integer representation on 16 bits (because of the truncation of 5 bits), we get:

$$N_y = \frac{y}{LSB_y} = \frac{1}{2^{-15} \cdot 2^5} = 2^{10} = 1024$$

and this corresponds to the output of the testbench.
The same computations can be made for all the other inputs to test their correctness.

5 Vivado report

Once I confirmed that everything was working properly based on the testbench results, I proceeded with synthesis and implementation on the Vivado platform, in order to analyze the power consumption, the maximum clock frequency and the resource utilization on the **Xilinx Zynq FPGA xc7z010clg400-1**.

5.1 Synthesis and Implementation warnings

After running the logic synthesis and the implementation, Vivado provided me some warnings:

Synthesis	Implementation	Summary Route Status
Status: Complete	Status: Complete	
Messages: 1 warning	Messages: 103 warnings	
Active run: synth_1_copy_1	Active run: impl_2	
Part: xc7z010clg400-1	Part: xc7z010clg400-1	
Strategy: Vivado Synthesis Defaults	Strategy: Vivado Implementation Defaults	
Report Strategy: Vivado Synthesis Default Reports	Report Strategy: Vivado Implementation Default Reports	
Incremental synthesis: Automatically selected checkpoint	Incremental implementation: None	

Figure 13: Warning messages

In particular, as shown in Figure 13, one regards the synthesis and the others the implementation. They are explained in detail in the following sections.

5.1.1 Synthesis warning explanation

As far as the synthesis warning is concerned, it is simply due to the fact the design is not large enough to benefit from parallel synthesis, thus the parallel flow will not be run.

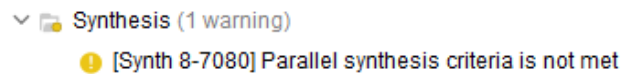


Figure 14: Synthesis warning

5.1.2 Implementation warnings explanation

Even for the implementation warnings, we should not be alarmed, as they are due to the **out-of-context synthesis mode** set within the synthesis options. This mode allows us to perform the implementation without mapping the I/O ports of the circuit to the FPGA physical pins.

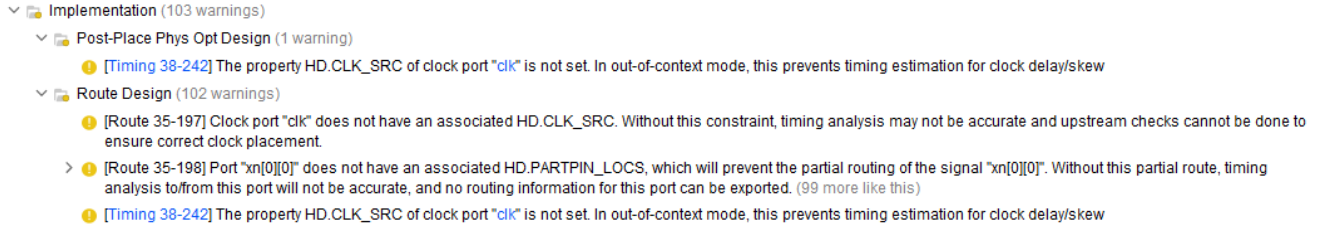


Figure 15: Implementation warnings

As illustrated in Figure 15, all the warnings are related to the lack of I/O port mapping.

5.2 Timing constraint and Critical Path

In order to be able to analyze the maximum achievable clock frequency and the critical paths of the circuit, I applied a clock constraint with a period $T = 15$ ns , which corresponds to a clock frequency $f = 66.667$ MHz. With this timing constraint, we get the following slacks:

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 1,341 ns	Worst Hold Slack (WHS): 0,273 ns	Worst Pulse Width Slack (WPWS): 7,000 ns
Total Negative Slack (TNS): 0,000 ns	Total Hold Slack (THS): 0,000 ns	Total Pulse Width Negative Slack (TPWS): 0,000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 215	Total Number of Endpoints: 215	Total Number of Endpoints: 394
All user specified timing constraints are met.		

Figure 16: Report Timing Summary

As we can see in Figure 16, the **Worst Negative Slack** (related to the Critical Path) is a positive value, therefore all the specified timing constraints are met successfully.

Moreover, we can straightforwardly compute the maximum achievable clock frequency as follows:

$$f_{max} = \frac{1}{T - WNS} = \frac{1}{15 - 1.341} \sim 73.21 \text{ MHz}$$

Furthermore, Vivado also provided the following table which illustrates what the **Critical Path** actually is:

Name	Slack ^{^1}	Levels	Routes	High Fanout	From	To	Total Delay	L
Path 1	1.341	14	14	27	g_DFF[0].DDF_x/do_s_reg[3]/C	Perc/g_DFF[0].PR_1/do_s_reg[16]/D	13.637	
Path 2	1.341	14	14	27	g_DFF[1].DDF_x/do_s_reg[3]/C	Perc/g_DFF[1].PR_1/do_s_reg[16]/D	13.637	
Path 3	1.341	14	14	27	g_DFF[2].DDF_x/do_s_reg[3]/C	Perc/g_DFF[2].PR_1/do_s_reg[16]/D	13.637	
Path 4	1.341	14	14	27	g_DFF[3].DDF_x/do_s_reg[3]/C	Perc/g_DFF[3].PR_1/do_s_reg[16]/D	13.637	
Path 5	1.341	14	14	27	g_DFF[4].DDF_x/do_s_reg[3]/C	Perc/g_DFF[4].PR_1/do_s_reg[16]/D	13.637	
Path 6	1.341	14	14	27	g_DFF[5].DDF_x/do_s_reg[3]/C	Perc/g_DFF[5].PR_1/do_s_reg[16]/D	13.637	
Path 7	1.341	14	14	27	g_DFF[6].DDF_x/do_s_reg[3]/C	Perc/g_DFF[6].PR_1/do_s_reg[16]/D	13.637	
Path 8	1.341	14	14	27	g_DFF[7].DDF_x/do_s_reg[3]/C	Perc/g_DFF[7].PR_1/do_s_reg[16]/D	13.637	
Path 9	1.341	14	14	27	g_DFF[8].DDF_x/do_s_reg[3]/C	Perc/g_DFF[8].PR_1/do_s_reg[16]/D	13.637	
Path 10	1.341	14	14	27	g_DFF[9].DDF_x/do_s_reg[3]/C	Perc/g_DFF[9].PR_1/do_s_reg[16]/D	13.637	

Figure 17: Table of Critical Paths

As displayed in Figure 17, there are 10 critical paths. These paths start from the barrier registers of the inputs x_i , pass through the Parallel Multipliers, and finally reach the respective pipeline registers downstream of the Multipliers.

We could have expected this right away, since the products are the most complex and slowest operations. In the following figure, these paths are highlighted in blue within the circuit:

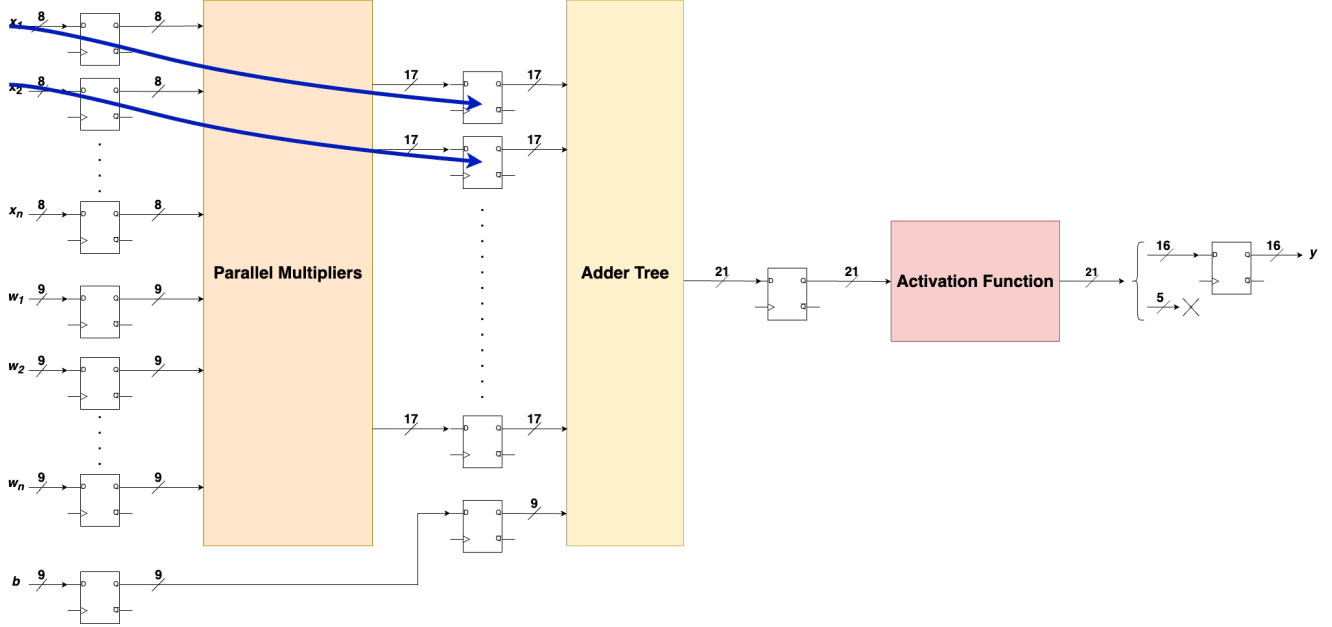


Figure 18: Critical Paths

5.3 Power Consumption

Regarding the power consumption of the implemented circuit, Vivado provided the following report:

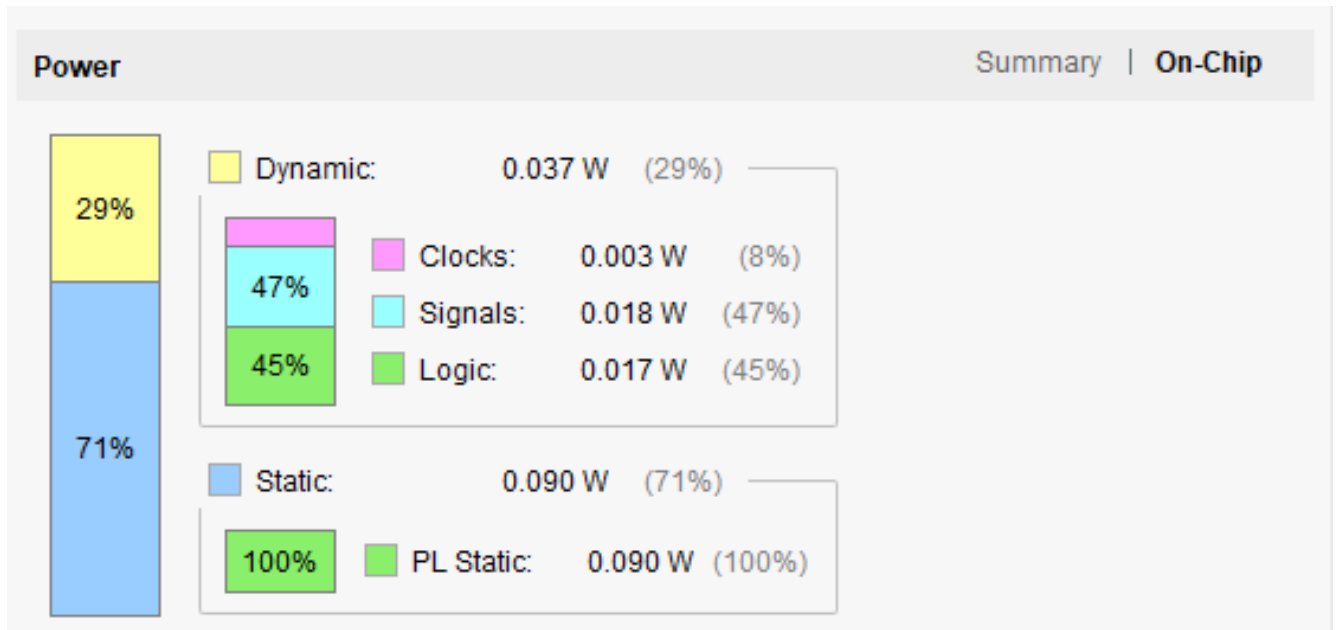


Figure 19: Power Consumption report

Figure 19 shows the percentages related to the static and dynamic power consumption of the circuit and their composition.

5.4 Resource Utilization

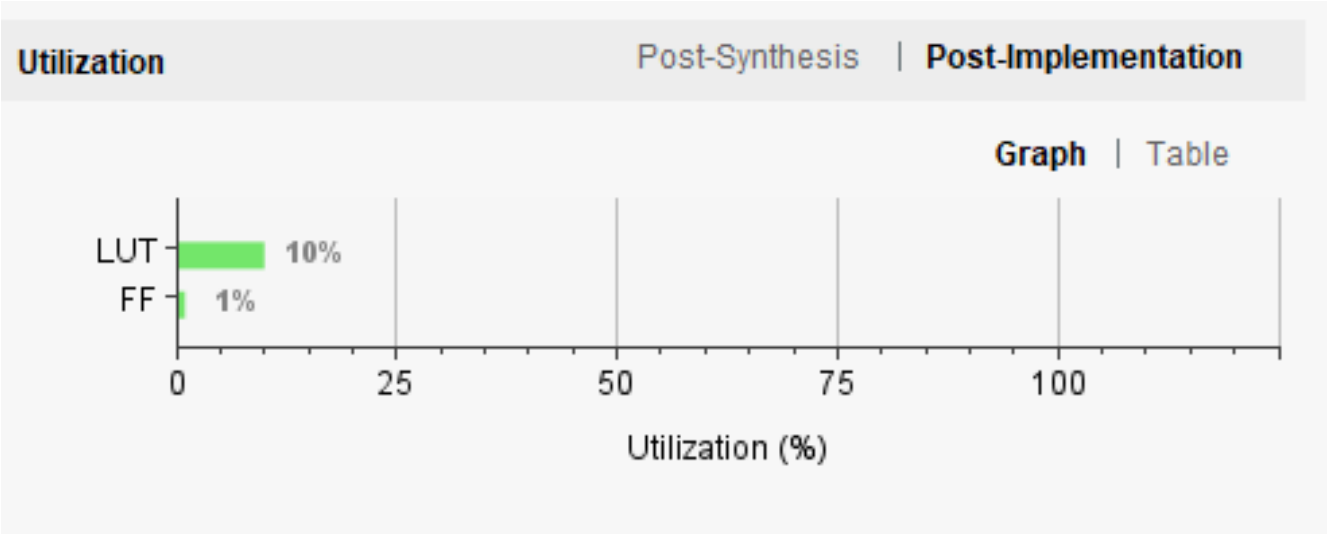


Figure 20: Resource Utilization report

Figure 20 illustrates the resource utilization of the circuit within the FPGA board.

6 Conclusion

In conclusion, this project has presented a possible implementation of the Perceptron on an FPGA platform, however, as explained in Section 1.2, there are many other potential solutions.

The one I chose offers a good **trade-off** between architectural complexity and performance in terms of maximum clock frequency, power consumption and resource utilization.

If the goal had been to reduce the power consumption and the resource utilization at the expense of the speed, one could have substituted the **Ripple Carry Adders** with **Serial Adders**, which have a simpler architecture, but are slower.

On the other hand, to speed up the circuit, one could have exploited the **Carry Look Ahead Adder** solution, which is faster, but more complex.

Finally, to further improve the maximum clock frequency, the critical paths can be split by adding additional pipeline registers along them.

In this case, they should be placed between the **Parallel Multiplier** stages.